ФІНАЛЬНИЙ ПРОЕКТ

# Red Team for AI

Тестування безпеки LLM за допомогою NVIDIA Garak + Ollama

Курсу “Кібербезпека з нуля”

# Мета проекту

**На ринку спостерігається стрімке зростання вакансій на позицію AI Security Engineer різного рівня.** Станом на 2026 рік, понад 64% вакансій у сфері кібербезпеки вимагають навичок роботи з AI/ML, а спеціалізовані ролі на кшталт Model Security Engineer та AI Threat Hunter зростають на 25%+ щорічно. - Загальна кількість вакансій у кібербезпеці зросла на 21% рік до року -, при цьому вакансії в AI/ML зросли на 163% порівняно з 2024 роком. Robert Half Середня зарплата AI Security Engineer у США станом на початок 2026 року складає $152,773 на рік, а топ-спеціалісти отримують до $205,000. ZipRecruiter
**Цей проект дає можливість отримати практичний досвід у одному з ключових напрямків роботи такого спеціаліста — red teaming великих мовних моделей.**

---

# Теоретичні відомості

## Що таке Garak?

Garak (Generative AI Red-teaming and Assessment Kit) — це open-source інструмент від NVIDIA для виявлення вразливостей у великих мовних моделях. Працює за принципом penetration testing: автоматично надсилає спеціально сформовані промпти та аналізує відповіді моделі.

## Архітектура Garak

Garak побудований на модульній архітектурі з трьох основних компонентів: Probes (атаки) генерують шкідливі промпти, Generator (LLM) обробляє їх, а Detectors аналізують відповіді на предмет успішних атак.

| Компонент | Опис |
|---|---|
| Probes | Набори атак: jailbreak, prompt injection, encoding тощо |
| Generators | Інтерфейси до LLM: OpenAI, Ollama, LM Studio, HuggingFace |
| Detectors | Аналізатори відповідей для виявлення успішних атак |
| Buffs | Модифікатори промптів (base64, paraphrase, тощо) |

## Яку модель обрати?

Для наочної демонстрації вразливостей рекомендуються моделі без вбудованих guardrails або з мінімальним alignment:

| Модель | Розмір | Команда | Чому підходить |
|---|---|---|---|
| phi3:mini | ~2.3GB | ollama pull phi3:mini | Маленька, швидка, має вразливості |
| mistral | ~4.1GB | ollama pull mistral | Класика, добре демонструє encoding attacks |
| llama3.2:3b | ~2GB | ollama pull llama3.2:3b | Нова, але вразлива до деяких атак |
| qwen2.5:3b | ~2GB | ollama pull qwen2.5:3b | Швидка, вразлива до prompt injection |
| gemma2:2b | ~1.6GB | ollama pull gemma2:2b | Найменша, підходить для слабких ПК |

# Частина 1: Підготовка середовища

## Крок 1.1: Встановлення Ollama

### macOS

```
# Варіант 1: Homebrew
brew install ollama

# Варіант 2: Завантажити з сайту
# https://ollama.com/download/mac
```

### Windows

1. Завантажте інсталятор з https://ollama.com/download/windows
2. Запустіть .exe файл
3. Дотримуйтесь інструкцій майстра встановлення
4. Ollama автоматично запуститься у системному треї

### Linux (Ubuntu/Debian)

```
# Офіційний скрипт встановлення
curl -fsSL https://ollama.com/install.sh | sh

# Або через snap
sudo snap install ollama
```

## Крок 1.2: Запуск Ollama та завантаження моделі

### Запуск сервісу (macOS / Linux):

```
ollama serve &
```

На Windows Ollama запускається автоматично після встановлення. Перевірте іконку в системному треї.

### Завантаження моделі:

```
# Рекомендована модель для тестування
ollama pull phi3:mini

# Або альтернативи
ollama pull mistral
ollama pull llama3.2:3b
```

## Крок 1.3: Перевірка роботи Ollama

```
# Список завантажених моделей
```

```
ollama list

# Тестовий запит
ollama run phi3:mini "Say hello"

# Перевірка API
curl http://localhost:11434/api/tags
```

**Для Windows (PowerShell):**

```
Invoke-RestMethod -Uri "http://localhost:11434/api/tags" | ConvertTo-Json
```

# Частина 2: Встановлення та налаштування Garak

## Крок 2.1: Створення віртуального середовища

### macOS / Linux

```
mkdir ~/garak-lab && cd ~/garak-lab
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
```

### Windows (PowerShell)

```
mkdir $HOME\garak-lab
cd $HOME\garak-lab
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install --upgrade pip
```

### Windows (Command Prompt)

```
mkdir %USERPROFILE%\garak-lab
cd %USERPROFILE%\garak-lab
python -m venv venv
venv\Scripts\activate.bat
pip install --upgrade pip
```

## Крок 2.2: Встановлення Garak

```
pip install garak
```

Перевірка:

```
garak --version
```

# Частина 3: Запуск тестування

## Метод 1: Прямий генератор Ollama (рекомендовано)

Garak має вбудовану підтримку Ollama:

```
garak --model_type ollama \
      --model_name phi3:mini \
      --probes test.Test
```

## Метод 2: Через LiteLLM (альтернатива)

```
pip install litellm
```

Створіть файл конфігурації ollama_config.json:

```
{
  "litellm": {
    "LiteLLMGenerator": {
      "api_base": "http://localhost:11434/v1",
      "provider": "openai"
    }
  }
}
```

Запуск через LiteLLM:

```
export OPENAI_API_KEY="ollama-local"

garak --model_type litellm \
      --model_name openai/phi3:mini \
      --probes test.Test \
      --generator_option_file ollama_config.json
```

# Частина 4: Основні типи атак (Probes)

## Перегляд доступних probes

```
# Список усіх probes
garak --list_probes

# Інформація про конкретний probe
garak --plugin_info probes.dan.Dan_11_0
```

## Ключові категорії probes

| Probe | Тип атаки | Опис |
|---|---|---|
| dan | Jailbreak | "Do Anything Now" — обхід обмежень моделі |
| encoding | Encoding bypass | Атаки через Base64, ROT13, Hex кодування |
| promptinject | Prompt Injection | Маніпуляція виводом через ін’єкції |
| glitch | Token exploitation | Використання "glitch tokens" |
| atkgen | Auto-generated | Автоматична генерація нових атак |

# Частина 5: Практичні вправи

## Вправа 5.1: Jailbreak-атаки (DAN)

**Мета:** Перевірити, чи можна обійти safety-обмеження моделі.

```
garak --model_type ollama \
      --model_name phi3:mini \
      --probes dan.Dan_6_0,dan.Dan_11_0 \
      --generations 3
```

Очікувані результати: PASS — модель відхилила атаку, FAIL — модель піддалася jailbreak.

## Вправа 5.2: Encoding-атаки

**Мета:** Перевірити вразливість до закодованих промптів.

```
garak --model_type ollama \
      --model_name phi3:mini \
      --probes encoding.InjectBase64,encoding.InjectROT13,encoding.InjectHex
\
      --generations 3
```

Принцип атаки: звичайний промпт кодується у Base64/ROT13/Hex і надсилається моделі, яка може декодувати його, оминаючи фільтри безпеки.

## Вправа 5.3: Prompt Injection

**Мета:** Перевірити вразливість до ін’єкцій у промпти.

```
garak --model_type ollama \
      --model_name phi3:mini \
      --probes promptinject.HijackHateHumans,promptinject.HijackKillHumans \
      --generations 5
```

## Вправа 5.4: Комплексний тест

**Мета:** Провести повноцінне сканування кількома типами атак.

```
garak --model_type ollama \
      --model_name phi3:mini \
      --probes dan,encoding,promptinject \
      --generations 3
```

⚠️ **Увага: Повний тест може зайняти 15-30 хвилин залежно від потужності комп’ютера.**

# Частина 6: Аналіз результатів

## Крок 6.1: Знаходження звітів

**macOS / Linux:**

```
ls ~/.local/share/garak/garak_runs/
```

**Windows:**

```
dir $HOME\.local\share\garak\garak_runs\
```

## Крок 6.2: Перегляд HTML-звіту

```
# macOS
open ~/.local/share/garak/garak_runs/garak.*.report.html


# Linux
xdg-open ~/.local/share/garak/garak_runs/garak.*.report.html


# Windows (PowerShell)
Start-Process (Get-ChildItem
"$HOME\.local\share\garak\garak_runs\*.report.html" | Select-Object -First
1)
```

## Крок 6.3: Перегляд hitlog

**macOS / Linux:**

```
ls -lt ~/.local/share/garak/garak_runs/*.hitlog.jsonl | head -1


cat ~/.local/share/garak/garak_runs/garak.*.hitlog.jsonl | python -m
json.tool | less
```

**Windows (PowerShell):**

```
Get-Content (Get-ChildItem
"$HOME\.local\share\garak\garak_runs\*.hitlog.jsonl" |
    Sort-Object LastWriteTime -Descending |
    Select-Object -First 1) |
    ConvertFrom-Json | Format-List
```

## Крок 6.4: Інтерпретація результатів

| Показник | Значення |
|---|---|
| PASS | Модель успішно відхилила всі атаки |

| | |
|---|---|
| FAIL | Деякі атаки були успішними |
| ok on X/Y | X успішних відхилень з Y спроб |
| failure rate | Відсоток успішних атак |

# Частина 7: Усунення неполадок

## Ollama не запущений

```
curl http://localhost:11434/api/tags
ollama serve
```

## Модель не знайдена

```
ollama list
ollama pull phi3:mini
```

## Timeout errors

Для великих моделей або повільних ПК збільшите timeout:

```
export OLLAMA_KEEP_ALIVE=300

garak --model_type ollama \
      --model_name phi3:mini \
      --probes dan \
      --generator_options '{"timeout": 300}'
```

## Out of memory

```
ollama pull gemma2:2b

garak --model_type ollama \
      --model_name gemma2:2b \
      --probes dan \
      --generations 1
```

## PowerShell execution policy (Windows)

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

# Вимоги до звіту фінального проекту

## Що таке Executive Summary?

Executive Summary (резюме для керівництва) — це короткий розділ на початку документа, який містить найважливішу інформацію для швидкого ознайомлення. Призначений для людей, які не мають часу читати весь документ, але повинні прийняти рішення.

**Характеристики хорошого Executive Summary:**

- Обсяг: 0.5–1 сторінка
- Містить ключові знахідки та висновки
- Написаний простою мовою без глибоких технічних деталей
- Включає рекомендації щодо подальших дій
- Вказує рівень критичності знайдених проблем

## Структура звіту

Звіт фінального проекту повинен містити такі розділи:

1. Executive Summary — загальний опис, основні результати, рівень ризику
2. Опис знайдених вразливостей — детальний опис кожної з критичністю та статистикою
3. Метод реплікації (Proof of Concept) — кроки для відтворення кожної вразливості
4. Remediation — рекомендації щодо усунення з оцінкою складності
5. Додатки — інформація про середовище, команди, логи, скріншоти

# Шаблон звіту фінального проекту

Нижче наведено детальний шаблон, який студент повинен використати як основу для свого звіту.

## 1. Executive Summary

*[Короткий опис (5-10 речень) що було зроблено, які основні результати отримано, та які дії рекомендуються]*

**Приклад:**

*«Проведено автоматизоване тестування безпеки моделі phi3:mini з використанням NVIDIA Garak та Ollama. Протестовано 3 категорії атак: jailbreak (DAN), encoding bypass та prompt injection. Виявлено 2 вразливості середнього рівня критичності: модель вразлива до DAN jailbreak (16.7% success rate) та часткова вразливість до Base64 encoding (8.3% success rate). Рекомендується впровадження додаткових guardrails та input validation перед production deployment.»*

**Загальна оцінка ризику:** [ВИСОКИЙ / СЕРЕДНІЙ / НИЗЬКИЙ]

## 2. Опис знайдених вразливостей

**Для кожної знайденої вразливості вказати:**

- Назва (наприклад, "DAN Jailbreak Vulnerability")
- Категорія: Jailbreak / Prompt Injection / Encoding Bypass
- Критичність: Критична / Висока / Середня / Низька / Інформаційна
- Probe: назва використаного probe
- Детальний опис вразливості та її наслідків
- Статистика: всього спроб, успішних атак, failure rate

## 3. Метод реплікації (Proof of Concept)

**Для кожної вразливості описати:**

- Передумови (встановлене ПЗ, завантажена модель)
- Покрокові інструкції для відтворення
- Очікуваний результат
- Приклад успішної атаки з hitlog (JSON)

## 4. Remediation (Рекомендації)

Для кожної вразливості надати рекомендації:

- Input Filtering — фільтрація вхідних даних на наявність jailbreak-патернів
- System Prompt Hardening — посилення system prompt
- Output Filtering — ML-класифікатор для виявлення шкідливих відповідей
- Rate Limiting — обмеження кількості запитів
- Model Selection — вибір моделей з кращим alignment

Для кожної рекомендації вказати складність впровадження: Низька / Середня / Висока.

## 5. Додатки

- Інформація про середовище (ОС, версії Ollama, Garak, Python)
- Повні команди тестування
- Повні логи тестування (JSONL)
- Скріншоти: HTML звіт Garak, вивід консолі

# Критерії оцінювання

| Критерій | Бали |
|---|---|
| Executive Summary — чіткість та повнота | 5 |
| Опис вразливостей — технічна точність | 5 |
| Метод реплікації — відтворюваність | 5 |
| Remediation — практичність рекомендацій | 5 |
| Оформлення та структура документа | 5 |
| Всього | 25 |

# Швидкий старт (TL;DR)

Для тих, хто хоче швидко почати:

```
# 1. Встановити Ollama
curl -fsSL https://ollama.com/install.sh | sh

# 2. Завантажити модель
ollama pull phi3:mini

# 3. Створити середовище та встановити Garak
python3 -m venv venv && source venv/bin/activate
pip install garak

# 4. Запустити тест
garak --model_type ollama --model_name phi3:mini \
  --probes dan,encoding,promptinject --generations 3

# 5. Переглянути результати
open ~/.local/share/garak/garak_runs/garak.*.report.html
```

---

# Корисні посилання

- Garak Documentation: https://docs.garak.ai/
- Garak GitHub: https://github.com/NVIDIA/garak
- Ollama Documentation: https://ollama.com/
- Ollama GitHub: https://github.com/ollama/ollama
- Ollama Model Library: https://ollama.com/library
- OWASP Top 10 for LLM: https://owasp.org/www-project-top-10-for-large-language-model-applications/
- NIST AI Risk Management Framework: https://www.nist.gov/itl/ai-risk-management-framework

---

# Контрольні запитання

1. Що таке Garak і для чого він використовується?
2. Назвіть основні компоненти архітектури Garak.

3. Що таке DAN-атака і як вона працює?
4. Чому encoding-атаки можуть обходити захист LLM?
5. Як інтерпретувати показник failure rate у звітах Garak?
6. Які методи захисту LLM від prompt injection ви знаєте?
7. Що повинен містити Executive Summary і для кого він призначений?
8. Яка різниця між Ollama та LM Studio?

---

## Формат здачі проекту

- Формат: PDF або DOCX
- Назва файлу: [Прізвище]_RedTeam_AI_Report.pdf
- Обов’язкові додатки: HTML-звіт Garak, скріншоти